

# Operating System: Unix

Thomas de Jaeger

[thomas.de-jaeger@epita.fr](mailto:thomas.de-jaeger@epita.fr)

# Class details

---

- 27h:  $2h \times 12$  + 1h mid term + 2h exam final
- Grade: 10% Participation  
40% Mid term exam  
50% Final exam
- For anonymous feedback: [Feedback - Google Docs](#)

# Class organisation

---

- Lectures
- Labs during class

# Syllabus

---

- **Session 1:** Introduction to Unix/Linux
- **Session 2:** Installation of Linux (WSL, VirtualBox, Mac)
- **Session 3:** Linux Architecture & File Systems
- **Session 4:** CLI Basics: Commands & Navigation
- **Session 5:** Advanced CLI: I/O & Text Processing
- **Session 6:** Shell Scripting I: Basics & Automation
- **Session 7:** Shell Scripting II: Advanced Scripts
- **Session 8:** Processes & Init Systems
- **Session 9:** Users, Groups & Permissions
- **Session 10:** Text Editors: vi/vim
- **Session 11:** Github ? Networking?
- **Session 12:** Remote Access, Downloads & Shell Customization

# Shell Scripting I: Basics & Automation

---

## Session 6

### **Objectives:**

- Know what a shell script is and how to write one
- Understand the use of the shebang (#!) line and permissions
- Be able to use variables, read, and echo
- Write conditional structures like if, else, and case
- Prepare for automation through scripting

# What is a shell script?

---

- A script is a plain text file containing commands designed to be run by Linux
- Executed sequentially by the shell
- Like a to-do list for your terminal!

**shell scripts include file manipulation, program execution, and printing text.**

**Example:**

```
#!/bin/  
#print Hello, word  
echo "Hello, world!«  
# show the date  
date
```

# Creating and running a shell script

---

Shell scripts have several required constructs that tell the shell environment what to do and when to do it.

**shebang**  `#!/bin/bash`  
`#print Hello, word`  
`echo "Hello, world!"`  
`# show the date`  
`date`

# #! shebang

---

**The seemingly insignificant #! characters at the beginning of a shell script has a major significance on how your script will be executed.**

The shebang is the combination of **the # (pound key) and ! (exclamation mark)**. It is used to specify the interpreter with which the given script will be run by default

e.g.:

#!/bin/bash -> interpreter should be bash shell

#!/bin/zsh -> interpreter should be Z shell

So it is important to precise the shell interpreter in the script as different shells have different syntax



# Creating and running a shell script

---

Shell scripts have several required constructs that tell the shell environment what to do and when to do it.

```
#           #!/bin/  
          #print Hello, word  
          echo "Hello, world!"  
          # show the date  
          date
```

All the lines starting by **#** (except the shebang) are **comments**, meaning that they are ignored

**However, they are very important to give explanations, notes for the collaborators**

# Creating and running a shell script

---

Shell scripts have several required constructs that tell the shell environment what to do and when to do it.

**command**  `#!/bin/  
#print Hello, word  
echo "Hello, world!"  
# show the date  
date`

Then you can write as many command lines as you want. Here, the script will print « Hello, world!»

# Creating and running a shell script

---

- Create a file with .sh extension:

-> gedit hello.sh

- Add commands with a shebang:

```
#!/bin/bash
```

```
#print Hello, word
```

```
echo "Hello, world!"
```

```
# show the date
```

```
date
```

- Save and exit
- Change the permission (session 9)

-> chmod +x hello.sh

- Run it

-> ./hello.sh, you can also run bash hello.sh and it will ignore the shebang

# Variables

---

- Syntax:

```
name="Thomas"  
echo "Welcome $name"
```

- NO SPACE BEFORE/AFTER =

- Refer with \$

```
#!/bin/sh  
MY_MESSAGE="Hello World"  
echo $MY_MESSAGE
```

- echo “The number of line is: \$(wc -l errors.txt)”

# Reading input from user

---

- Syntax:

```
#!/bin/bash
echo "What is your name?"
read MY_NAME
echo "Hello $MY_NAME - hope you're well."
```

- **Read** stores input into a variable
- Use **export** if the variable needs to be available in sub-shells or other script  
export MY\_NAME

# Reading input from user

---

- Example myvar.sh:

```
#!/bin/sh
echo "MYVAR is: $MYVAR"
MYVAR="hi there"
echo "MYVAR is: $MYVAR"
```

What is the output, if you run ./myvar.sh ?

# Reading input from user

---

- Example myvar.sh:

```
#!/bin/sh  
echo "MYVAR is: $MYVAR"  
MYVAR="hi there"  
echo "MYVAR is: $MYVAR"
```

What is the output, if you run ./myvar.sh ?

MYVAR is:

MYVAR is: hi there

# Reading input from user

---

- Now try:

```
$ MYVAR="hello"
```

```
$ ./myvar.sh
```



# Reading input from user

---

- Now try:

```
$ MYVAR="hello"
```

```
$ ./myvar.sh
```

Output:

```
MYVAR is:
```

```
MYVAR is: hi there
```

It is the same, because when you run `./myvar.sh`, a new shell is used

You can use `export`:

```
$ MYVAR="hello"
```

```
$ export MYVAR
```

```
$ ./myvar.sh
```

# Reading input from user

---

You can use export:

```
$ MYVAR=hello
```

```
$ export myvar
```

```
$ ./myvar.sh
```

Output:

```
MYVAR is: hello
```

```
MYVAR is: hi there
```

If you do: `$ echo MYVAR` you will see hello

**Once the shell script exits, its environment is destroyed. But MYVAR keeps its value of hello within your interactive shell.**

# Reading input from user

---

```
#!/bin/bash  
echo "What is your name?"  
read USER_NAME  
echo "Hello $USER_NAME"  
echo "I will create you a file called $USER_NAME.txt"  
touch $USER_NAME.txt
```

What will happen?

# Reading input from user

---

What can we do?

# Curly brackets

---

What can we do? **CURLY brackets**

- Curly braces {} help clarify variable boundaries
- Prevent confusion when variables are next to other characters

```
#!/bin/bash
echo "What is your name?"
read USER_NAME
echo "Hello $USER_NAME"
echo "I will create you a file called ${USER_NAME}.txt"
touch "${USER_NAME}.txt"
```

# Conditions if/else

---

## Structure:

```
if [ ... ];then  
    # if-code  
else  
    # else-code  
fi
```

## Common numeric test operators:

- eq (equal)
- ne (not equal)
- gt (greater than)
- lt (less than)
- ge (greater or equal)
- le (less or equal)

## Common file test operators:

- f file → is a file
- d dir → is a directory
- x file → is executable
- r file → is readable
- w file → is writable

# Examples

---

**Check if a myfile.txt exists:**

# Examples

---

## **Check if a file exists:**

```
if [ -f myfile.txt ]; then  
    echo "File exists."  
else  
    echo "File not found."  
fi
```



# Examples

---

## **Check if a file exists:**

```
#!/bin/bash
if [ -f myfile.txt ]; then
    echo "File exists."
else
    echo "File not found."
fi
```

## **Compare two numbers (a,b):**

# Examples

---

## **Check if a file exists:**

```
#!/bin/bash
if [ -f myfile.txt ]; then
    echo "File exists."
else
    echo "File not found."
fi
```

## **Compare two numbers:**

```
#!/bin/bash
echo "Enter two numbers:"
read a b
if [ $a -eq $b ]; then
    echo "Both are equal"
elif [ $a -gt $b ]; then
    echo "$a is greater than $b"
else
    echo "$b is greater than $a"
fi
```

# Examples

---

## **Check if a file exists:**

```
#!/bin/bash
if [ -f myfile.txt ]; then
    echo "File exists."
else
    echo "File not found."
fi
```

## **Compare two numbers:**

```
#!/bin/bash
echo "Enter two numbers:"
read a b
if [ $a -eq $b ]; then
    echo "Both are equal"
elif [ $a -gt $b ]; then
    echo "$a is greater than $b"
else
    echo "$b is greater than $a"
fi
```

## **Ask for username and check if it is root:**

# Examples

---

## Check if a file exists:

```
#!/bin/bash
if [ -f myfile.txt ]; then
    echo "File exists."
else
    echo "File not found."
fi
```

## Compare two numbers:

```
#!/bin/bash

echo "Enter two numbers:"
read a b
if [ $a -eq $b ]; then
    echo "Both are equal"
elif [ $a -gt $b ]; then
    echo "$a is greater than $b"
else
    echo "$b is greater than $a"
fi
```

## Ask for username and check if it is root:

```
#!/bin/bash
echo "Enter your username:"
read user
if [ "$user" == "root" ]; then
    echo "You are the administrator."
else
    echo "You are a regular user."
fi
```

# Case structure

---

**The case structure is perfect when you have many conditions to evaluate. Instead of chaining multiple if and elif, you define patterns in a clean and readable block.**

- Start with case
- Patterns use | for alternates
- Every block ends with ;;
- esac closes the case block

# Case structure

---

## Exemples.

- Give a number between 1 and 3
- Print “your number is one” if it is 1, etc....

```
echo "Enter a number (1-3):"
```

```
read choice
```

```
if [ "$choice" -eq 1 ]; then
```

```
    echo "You chose one."
```

```
elif [ "$choice" -eq 2 ]; then
```

```
    echo "You chose two."
```

```
elif [ "$choice" -eq 3 ]; then
```

```
    echo "You chose three."
```

```
else
```

```
    echo "Invalid choice."
```

```
fi
```

# Case structure

## Examples.

- Give a number between 1 and 3
- Print “your number is one” if it is 1, etc....

```
echo "Enter a number (1-3):"  
read choice  
if [ "$choice" -eq 1 ]; then  
    echo "You chose one."  
elif [ "$choice" -eq 2 ]; then  
    echo "You chose two."  
elif [ "$choice" -eq 3 ]; then  
    echo "You chose three."  
else  
    echo "Invalid choice."  
fi
```



`if` is used for conditional logic; `case` is cleaner for multiple value comparisons. Use `case` when comparing a single variable to many values

```
echo "Enter a number from 1 to 3:"  
read choice  
  
case $choice in  
    1) echo "You chose one.;;"  
    2) echo "You chose two.;;"  
    3) echo "You chose three.;;"  
    *) echo "Invalid choice.;;"  
esac
```

# What you learn:

---

## **You now know:**

- How to write and run scripts
- Use variables and input
- Create decisions using if and case
- Apply comments and use the `#!` line:

**Let's practice!!**